

Cryptanalysis and Security Analysis: Attack & Evaluate

1. Project Overview

This project explores the fascinating world of cryptanalysis by practically attacking intentionally vulnerable cryptographic implementations. The goal is to deeply understand why certain systems fail by building both the vulnerable cipher and the attack script that breaks it.

We focus on three distinct areas of cryptography:

1. **Classical Cryptography:** The Vigenère Cipher (Frequency Analysis).
2. **Modern Asymmetric Cryptography:** A Toy RSA Implementation (Factoring attacks).
3. **Implementation Security:** Early-exit string comparisons (Side-Channel Timing attacks).

For each area, this project:

- Implements the vulnerable system.
 - Implements the mathematical or side-channel attack to break it.
 - Measures and evaluates the security strength and attack cost.
 - Implements the industry-standard mitigation to prove the vulnerability is resolved.
-

2. Detailed Breakdown of the Systems

2.1 The Vigenère Cipher (Classical)

- **What it is:** A polyalphabetic substitution cipher. Instead of shifting every letter by the same amount (like the Caesar cipher), it uses a keyword to shift different letters by different amounts.
- **The Vulnerability:** The Vigenère cipher does not destroy the underlying frequency distribution of the language; it simply splits it across k different Caesar ciphers, where k is the length of the key.
- **The Attack (Index of Coincidence & Chi-Squared):**
 1. We guess the key length k by splitting the ciphertext into k columns and calculating the **Index of Coincidence (IC)** for each column. If k is correct, each column is a simple Caesar cipher, and its IC will match the IC of standard English (~ 0.066).
 2. Once k is known, we test all 26 possible shifts for each of the k columns. We use the **Chi-Squared statistic** to compare the resulting frequency distribution of the decrypted column against known English letter frequencies. The shift with the lowest Chi-Squared score is almost certainly the correct letter of the key.
- **How to Verify:** Run `python3 experiments/exp1_vigenere.py`. The script will encrypt thousands of random slices of English text and successfully recover the exact key with near 100% accuracy as long as the ciphertext is sufficiently long.

2.2 Toy RSA (Asymmetric)

- **What it is:** A miniature version of the RSA public-key cryptosystem. It generates two prime numbers p and q , computes $N = p \times q$, and uses N as the public modulus.
- **The Vulnerability:** The security of RSA relies entirely on the difficulty of factoring N back into p and q . If N is too small, it can be factored easily.
- **The Attack (Pollard's Rho):** While Trial Division and Fermat's Factorization are slow, **Pollard's Rho algorithm** uses cycle-finding (similar to Floyd's tortoise and hare) to factor numbers in roughly $O(N^{1/4})$ time.
- **How to Verify:** Run `python3 experiments/exp2_rsa.py`. You will see Pollard's Rho factor a 48-bit RSA key in a fraction of a second. Extrapolating the log-linear curve proves that a 2048-bit key would take trillions of years to factor using this same method, demonstrating why large keys are secure.

2.3 Timing Side Channels (Implementation)

- **What it is:** A secure authentication system comparing a user's guess against a secret string.
 - **The Vulnerability:** The naive comparison (`vulnerable_check.py`) compares the strings byte-by-byte and returns `False` immediately upon finding the first mismatch. This creates a side-channel: a guess that is completely wrong returns faster than a guess that gets the first few characters correct.
 - **The Attack:** We write an oracle attack that measures the execution time of the comparison function using `time.perf_counter_ns()`. By guessing each possible character for the first position and repeating the test N times (to filter out system noise), the correct character will consistently take slightly longer to evaluate. We recover the secret character-by-character.
 - **How to Verify:** Run `python3 experiments/exp3_timing.py`. The script will accurately extract an 8-character secret out of a search space of 36^8 possibilities by only making a few thousand oracle queries.
-

3. Mitigations

Understanding how to break these systems naturally leads to understanding how to fix them.

1. **Vigenère Mitigation:** By making the key completely random and exactly as long as the plaintext, the Vigenère cipher becomes a **One-Time Pad**, which is mathematically unbreakable. The character frequencies become perfectly uniform, rendering Chi-Squared attacks useless.
 2. **RSA Mitigation:** Increase the modulus size to at least 2048 bits and employ standard padding schemes like **OAEP** (Optimal Asymmetric Encryption Padding) to prevent structural attacks.
 3. **Timing Mitigation:** Use a **constant-time comparison function** (like `hmac.compare_digest` in Python). This function always compares every single byte before returning, regardless of whether a mismatch was found early on. Running `python3 experiments/exp4_mitigations.py` demonstrates that against a constant-time check, the timing attack's accuracy drops to pure random chance ($\sim 2.7\%$).
-

4. How to Use and Evaluate This Project

The project is structured to be entirely reproducible on your local machine.

Repository Layout

- `ciphers/`: Contains the implementations of Vigenère, Toy RSA, and the string checkers.
- `attacks/`: Contains the Python algorithms used to break the ciphers.
- `experiments/`: Contains the scripts that run thousands of trials to gather statistical data on attack success rates.
- `plots/`: Contains the visual graphs generated from the experiment data.

Step-by-Step Evaluation

1. **Run the Live Demo** Open your terminal in the `crypto` folder and run:

```
python3 demo.py
```

This will execute a live, step-by-step demonstration of the attacks breaking the ciphers in real-time right before your eyes.

2. **Regenerate the Data** If you want to verify the statistical integrity of the attacks (e.g., proving that Vigenère needs ~ 1000 characters to achieve 100% success), run:

```
python3 run_all.py
```

This command orchestrates everything: it runs the unit tests, executes all four intensive experiments, saves the resulting data to the `data/results/` folder as CSV files, and instructs `matplotlib` to redraw all the graphs in the `plots/` folder.

3. **Review the Plots** Open the `plots/` folder.

- `vigenere_success.png`: Shows how attack accuracy correlates with ciphertext length.
- `rsa_factoring.png`: Visually demonstrates why Pollard's Rho is superior to Trial Division and why key lengths must scale exponentially.
- `timing_attack.png`: Compares the 100% success rate of the timing attack against the vulnerable function versus the 0% success rate against the mitigated, constant-time function.

Final Thoughts

This project successfully transitions cryptography from theoretical mathematics into practical engineering. By building, attacking, and evaluating these systems, we observe firsthand that secure algorithms are only as strong as their mathematical foundations (RSA) and their implementation details (Timing Attacks).